Simulador de TCC Online no GitHub

https://gemini.google.com/app/d41e00edd872827b

User prompt: com o TCC pronto, como eu poderia criar a simulação do sistema que apresentamos de forma online em um repo do Github?

Response: Para disponibilizar online a simulação do Sistema Tutor Inteligente apresentada no seu TCC, a melhor estratégia de Engenharia de Software e MLOps[cite: 1, 5] consiste em criar um repositório no GitHub integrado a uma interface interativa (web app) em Python hospedada gratuitamente, combinada com automação via GitHub Actions.

1. Estratégia de Arquitetura e Hospedagem Online

Para rodar a simulação online sem custos e direto pelo repositório:

1. **Repositório GitHub (Controle de Versão e CI/CD)**: Hospeda o código-fonte, configurações de dependências, documentação metodológica e rotinas de automação.
2. **Streamlit Community Cloud ou Hugging Face Spaces**: Ambas as plataformas conectam-se diretamente ao seu repositório no GitHub. Cada `git push` atualiza automaticamente a aplicação web pública, gerando um link ativo para a banca ou avaliadores testarem.
3. **GitHub Pages (Visualização Complementar)**: Se optar por exportar gráficos estáticos ou relatórios interativos gerados por bibliotecas como Plotly, o GitHub Pages pode servir de vitrine de documentação.

2. Estrutura Recomendada do Repositório

```
sti-qlearning-simulation/
├── .github/
│   └── workflows/
│       └── ci.yml                # Pipeline de testes e integridade do modelo
├── src/
│   ├── __init__.py
│   ├── environment.py            # Ambiente estocástico do aluno (MDP: S, A, P
│   ├── agent.py                  # Agente Q-learning (Tabela Q, alpha, gamma,
│   └── simulator.py              # Loop de 500 episódios e coleta de métricas
├── app.py                        # Interface web interativa (Streamlit)
├── requirements.txt              # Dependências mínimas
├── README.md                     # Documentação acadêmica e instruções de exec
└── LICENSE                       # Licença de código aberto (ex: MIT)
```

3. Código-Fonte Essencial da Simulação

Abaixo estão os módulos fundamentais respeitando rigorosamente a parametrização do Capítulo 4 do seu TCC ($|S| = 5$, $|A| = 10$, $\alpha = 0,20$, $\gamma = 0,90$, $\epsilon_0 = 1,00$, $\epsilon_{\min} = 0,05$ e decaimento $0,985$):

A. Ambiente e Agente (`src/environment.py` e `src/agent.py`)

```
# src/environment.py
import numpy as np

class StudentEnvironment:
```

```
    """
    Ambiente simulado do estudante baseado em Processos de Decisão Markovianos.
    Estados: E0 (Ruim), E1 (Regular), E2 (Bom), E3 (M. Bom), E4 (Excelente).
    Ações: A0 a A9 (Revisões, Exemplos, Listas de Fixação, Problemas e Desafios)
    """
    def __init__(self):
        self.state_labels = ["E0 (Ruim)", "E1 (Regular)", "E2 (Bom)", "E3 (M. Bo
        self.rewards = [1, 3, 5, 7, 10]
        self.current_state = 0  # Inicia em E0

    def reset(self):
        self.current_state = 0
        return self.current_state

    def step(self, action):
        # Matriz de transição pedagógica alinhada à Zona de Desenvolvimento Prox
        # Ações ideais por estado: E0->A0, E1->A2, E2->A4, E3->A7, E4->A9
        ideal_actions = {0: 0, 1: 2, 2: 4, 3: 7, 4: 9}
        s = self.current_state

        if action == ideal_actions[s]:
            # Ação pedagógica ótima: alta chance de progressão
            probs = np.zeros(5)
            if s < 4:
                probs[s + 1] = 0.75
                probs[min(s + 2, 4)] = 0.15
                probs[s] = 0.10
            else:
                probs[4] = 1.00
        else:
            # Ação inadequada: chance de estagnação ou regressão cognitiva
            probs = np.zeros(5)
            probs[max(0, s - 1)] = 0.40
            probs[s] = 0.50
            probs[min(s + 1, 4)] = 0.10

        self.current_state = int(np.random.choice(5, p=probs))
        reward = self.rewards[self.current_state]
        return self.current_state, reward

# src/agent.py
import numpy as np

class QLearningTutor:
    """
    Agente Pedagógico baseado no algoritmo Q-learning com exploração epsilon-gre
    """
    def __init__(self, n_states=5, n_actions=10, alpha=0.20, gamma=0.90,
                 epsilon=1.00, epsilon_min=0.05, decay=0.985):
        self.n_states = n_states
        self.n_actions = n_actions
        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon
        self.epsilon_min = epsilon_min
```

```
        self.decay = decay
        self.q_table = np.zeros((n_states, n_actions))

    def choose_action(self, state):
        if np.random.rand() < self.epsilon:
            return np.random.randint(self.n_actions)  # Exploração
        return int(np.argmax(self.q_table[state, :]))  # Explotação

    def update(self, s, a, r, next_s):
        best_next_a = np.argmax(self.q_table[next_s, :])
        td_target = r + self.gamma * self.q_table[next_s, best_next_a]
        td_error = td_target - self.q_table[s, a]
        self.q_table[s, a] += self.alpha * td_error
        return abs(td_error)

    def decay_epsilon(self):
        self.epsilon = max(self.epsilon_min, self.epsilon * self.decay)
```

B. Aplicação Web Interativa (`app.py`)

```
import streamlit as st
import numpy as np
import pandas as pd
from src.environment import StudentEnvironment
from src.agent import QLearningTutor

st.set_page_config(page_title="Simulador STI - Q-Learning", layout="wide")

st.title("🎓 Sistema Tutor Inteligente Adaptativo (Q-Learning)")
st.markdown("Simulador computacional desenvolvido para o TCC da Pós-Graduação em

# Barra lateral com parâmetros
st.sidebar.header("Hiperparâmetros do Modelo")
episodes = st.sidebar.slider("Episódios", 50, 1000, 500, step=50)
alpha = st.sidebar.slider("Taxa de Aprendizado (α)", 0.05, 0.50, 0.20, step=0.05
gamma = st.sidebar.slider("Fator de Desconto (γ)", 0.50, 0.99, 0.90, step=0.01)

if st.sidebar.button("Executar Simulação"):
    env = StudentEnvironment()
    agent = QLearningTutor(alpha=alpha, gamma=gamma)

    rewards_history = []
    td_errors_history = []

    progress_bar = st.progress(0)
    for ep in range(episodes):
        state = env.reset()
        ep_reward = 0
        ep_td_error = 0

        for step in range(20):
            action = agent.choose_action(state)
            next_state, reward = env.step(action)
            err = agent.update(state, action, reward, next_state)
```

```
            ep_reward += reward
            ep_td_error += err
            state = next_state

        agent.decay_epsilon()
        rewards_history.append(ep_reward / 20.0)
        td_errors_history.append(ep_td_error / 20.0)
        progress_bar.progress((ep + 1) / episodes)

    st.success("Simulação concluída!")

    col1, col2 = st.columns(2)
    with col1:
        st.subheader("Curva de Recompensa Média por Passo")
        st.line_chart(pd.DataFrame({"Recompensa": rewards_history}))
    with col2:
        st.subheader("Evolução do Erro TD Médio")
        st.line_chart(pd.DataFrame({"Erro TD": td_errors_history}))

    st.subheader("Tabela Q Convergida (Valores Q*(s, a))")
    q_df = pd.DataFrame(
        agent.q_table,
        index=["E0 (Ruim)", "E1 (Regular)", "E2 (Bom)", "E3 (M. Bom)", "E4 (Exce
        columns=[f"A{i}" for i in range(10)]
    )
    st.dataframe(q_df.style.highlight_max(axis=1, color="lightgreen"))
```

C. Arquivos de Suporte (`requirements.txt` e `.github/workflows/ci.yml`)

`requirements.txt`:

```
streamlit>=1.30.0
numpy>=1.24.0
pandas>=2.0.0
pytest>=7.0.0
```

`.github/workflows/ci.yml` (Garantindo governança de MLOps no repositório):

```
name: Testes de Integridade do Modelo

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v3
    - name: Configurar Python
      uses: setup-python@v4
      with:
        python-version: '3.10'
    - name: Instalar Dependências
      run: |
        python -m pip install --upgrade pip
```

```
        pip install -r requirements.txt
    - name: Executar Testes Unitários de Convergência
      run: |
        python -c "from src.agent import QLearningTutor; tutor = QLearningTutor(
```

4. Passo a Passo para Publicação Online

1. **Subir os arquivos no GitHub**:
    - Crie um repositório chamado `sti-qlearning-tcc`.
    - Inicialize o Git localmente e faça o primeiro commit:

      ```
      git init
      git add .
      git commit -m "feat: simulador interativo q-learning do tcc"
      git branch -M main
      git remote add origin https://github.com/SEU_USUARIO/sti-qlearning-tcc.
      git push -u origin main
      ```

2. **Deploy Gratuito na Nuvem (Streamlit Community Cloud)**:
    - Acesse `share.streamlit.io` e faça login com a sua conta do GitHub.
    - Clique em **"New app"**.
    - Selecione o repositório `sti-qlearning-tcc`, a branch `main` e indique o arquivo principal como `app.py`.
    - Clique em **"Deploy"**. Em menos de 2 minutos, a plataforma gerará uma URL pública (ex.: `[https://sti-qlearning-tcc.streamlit.app](https://sti-qlearning-tcc.streamlit.app)`).

Você pode anexar esse link diretamente no `README.md` do repositório e utilizá-lo durante a apresentação da sua defesa. Deseja que eu elabore o modelo completo do arquivo `README.md` acadêmico para o repositório?

User prompt: vamos usar o Github e o Githubpages e vamos iniciar o projeto localmente com o Antigravity programar com o Antigravity IDE

Response: Para viabilizar a execução da simulação do Sistema Tutor Inteligente (STI) diretamente no **GitHub Pages**, precisamos atentar para um detalhe crucial de arquitetura de software: o GitHub Pages hospeda exclusivamente **páginas e arquivos estáticos (HTML, CSS e JavaScript)**, não executando servidores de backend em Python nativo.

Para manter a fidelidade matemática do seu TCC (ambientes markovianos, matriz de transição estocástica, cálculo do Erro TD e atualização da Tabela Q), a solução de engenharia mais limpa e moderna consiste em implementar o motor do Q**-learning diretamente no frontend com JavaScript nativo** (ou compilado) e bibliotecas de gráficos (como Chart.js ou Plotly.js). Dessa forma, o simulador roda 100% no navegador do usuário final (banca examinadora ou alunos), com zero custo de servidor e deploy contínuo via GitHub Actions ou branch `gh-pages`.

Abaixo está o roteiro completo de estruturação, codificação e publicação para você programar no seu ambiente local (Antigravity IDE / VS Code) e subir ao GitHub.

1. Estrutura de Diretórios do Projeto Local

No seu ambiente local, organize os arquivos da seguinte forma:

```
sti-qlearning-simulation/
├── .github/
│   └── workflows/
│       └── deploy.yml        # Automação de deploy para o GitHub Pages (CI/CD)
├── index.html                # Interface gráfica do usuário (dashboard com cont
├── style.css                 # Estilização visual (design limpo e responsivo)
├── qlearning.js              # Agente Q-learning e ambiente do estudante (lógic
├── README.md                 # Documentação técnica e acadêmica do projeto
└── LICENSE                   # Licença de código aberto (ex: MIT)
```

2. Implementação Técnica dos Arquivos

A. O Motor de Simulação (`qlearning.js`)

Este script transpõe fielmente o modelo do Capítulo 4 do seu TCC: 5 estados cognitivos (E_0 a E_4), 10 ações pedagógicas (A_0 a A_9), recompensas fixas ($r \in [1, 3, 5, 7, 10]$), taxa de aprendizado $\alpha = 0{,}20$, fator de desconto $\gamma = 0{,}90$ e decaimento exponencial de exploração ϵ de $1{,}00$ até $0{,}05$.

```
// qlearning.js - Motor do Agente Q-Learning e Ambiente Estocástico do Estudante

class StudentEnvironment {
    constructor() {
        this.states = ["E0 (Ruim)", "E1 (Regular)", "E2 (Bom)", "E3 (M. Bom)", "
        this.rewards = [1, 3, 5, 7, 10];
        this.currentState = 0; // Inicia em E0
        // Ações ótimas parametrizadas no TCC (Zona de Desenvolvimento Proximal)
        // E0->A0 (Revisão base), E1->A2 (Exemplo), E2->A4 (Fixação), E3->A7 (Pr
        this.idealActions = { 0: 0, 1: 2, 2: 4, 3: 7, 4: 9 };
    }

    reset() {
        this.currentState = 0;
        return this.currentState;
    }

    step(action) {
        const s = this.currentState;
        const rand = Math.random();

        if (action === this.idealActions[s]) {
            // Ação compatível com o estado cognitivo: alta chance de progressão
            if (s < 4) {
                if (rand < 0.75) this.currentState = s + 1;
                else if (rand < 0.90) this.currentState = Math.min(s + 2, 4);
                else this.currentState = s;
            } else {
                this.currentState = 4;
            }
```

```
        } else {
            // Ação pedagógica descalibrada: risco de estagnação ou regressão
            if (rand < 0.40) this.currentState = Math.max(0, s - 1);
            else if (rand < 0.90) this.currentState = s;
            else this.currentState = Math.min(s + 1, 4);
        }

        const reward = this.rewards[this.currentState];
        return { nextState: this.currentState, reward: reward };
    }
}

class QLearningAgent {
    constructor(alpha = 0.20, gamma = 0.90, epsilon = 1.00, epsilonMin = 0.05, d
        this.nStates = 5;
        this.nActions = 10;
        this.alpha = alpha;
        this.gamma = gamma;
        this.epsilon = epsilon;
        this.epsilonMin = epsilonMin;
        this.decay = decay;

        // Inicializa a Tabela Q (5 x 10) com zeros
        this.qTable = Array.from({ length: this.nStates }, () => Array(this.nAct
    }

    chooseAction(state) {
        // Dilema Exploration vs. Exploitation (Epsilon-Greedy)
        if (Math.random() < this.epsilon) {
            return Math.floor(Math.random() * this.nActions); // Exploração alea
        }
        // Explotação: seleciona o índice de ação com maior utilidade Q*(s, a)
        const row = this.qTable[state];
        return row.indexOf(Math.max(...row));
    }

    update(s, a, r, nextS) {
        const bestNextA = Math.max(...this.qTable[nextS]);
        const tdTarget = r + this.gamma * bestNextA;
        const tdError = tdTarget - this.qTable[s][a];
        this.qTable[s][a] += this.alpha * tdError;
        return Math.abs(tdError);
    }

    decayEpsilon() {
        this.epsilon = Math.max(this.epsilonMin, this.epsilon * this.decay);
    }
}
```

B. A Interface Web (`index.html`)

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
```

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Simulador STI - Q-Learning (TCC Pós-IA)</title>
    <link rel="stylesheet" href="style.css">
    <!-- Chart.js para renderização de curvas de treinamento -->
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
    <header>
        <h1>Simulador de Sistema Tutor Inteligente (Q-Learning)</h1>
        <p>Modelo Adaptativo baseado em Processos de Decisão Markovianos | Facul
    </header>

    <main class="container">
        <section class="controls">
            <h2>Parâmetros da Simulação</h2>
            <div class="control-group">
                <label for="episodes">Episódios:</label>
                <input type="number" id="episodes" value="500" min="50" max="200
            </div>
            <div class="control-group">
                <label for="alpha">Taxa de Aprendizado (&alpha;):</label>
                <input type="number" id="alpha" value="0.20" step="0.05" min="0.
            </div>
            <div class="control-group">
                <label for="gamma">Fator de Desconto (&gamma;):</label>
                <input type="number" id="gamma" value="0.90" step="0.01" min="0.
            </div>
            <button id="runBtn" onclick="runSimulation()">Executar Simulação</bu
            <div id="status" class="status">Pronto para rodar.</div>
        </section>

        <section class="charts-grid">
            <div class="chart-card">
                <h3>Curva de Recompensa Média por Passo</h3>
                <canvas id="rewardChart"></canvas>
            </div>
            <div class="chart-card">
                <h3>Evolução do Erro TD Médio</h3>
                <canvas id="tdErrorChart"></canvas>
            </div>
        </section>

        <section class="results">
            <h2>Tabela Q Convergida ($Q^*(s, a)$)</h2>
            <div class="table-responsive">
                <table id="qTableDisplay">
                    <!-- Tabela gerada dinamicamente via JS -->
                </table>
            </div>
        </section>
    </main>

    <script src="qlearning.js"></script>
```

```
<script>
    let rewardChart, tdErrorChart;

    function runSimulation() {
        const episodes = parseInt(document.getElementById('episodes').value)
        const alpha = parseFloat(document.getElementById('alpha').value);
        const gamma = parseFloat(document.getElementById('gamma').value);
        const statusDiv = document.getElementById('status');

        statusDiv.innerText = "Simulando 500 episódios... Aguarde.";

        const env = new StudentEnvironment();
        const agent = new QLearningAgent(alpha, gamma);

        const rewardData = [];
        const tdErrorData = [];
        const labels = [];

        for (let ep = 1; ep <= episodes; ep++) {
            let s = env.reset();
            let epReward = 0;
            let epError = 0;

            for (let step = 0; step < 20; step++) {
                const a = agent.chooseAction(s);
                const { nextState, reward } = env.step(a);
                const err = agent.update(s, a, reward, nextState);

                epReward += reward;
                epError += err;
                s = nextState;
            }

            agent.decayEpsilon();

            if (ep % 5 === 0 || ep === 1 || ep === episodes) {
                labels.push(ep);
                rewardData.push((epReward / 20).toFixed(2));
                tdErrorData.push((epError / 20).toFixed(2));
            }
        }

        statusDiv.innerText = `Simulação concluída com sucesso! Recompensa f
        renderCharts(labels, rewardData, tdErrorData);
        renderTable(agent.qTable, env.states);
    }

    function renderCharts(labels, rewards, errors) {
        if (rewardChart) rewardChart.destroy();
        if (tdErrorChart) tdErrorChart.destroy();

        const ctxR = document.getElementById('rewardChart').getContext('2d')
        rewardChart = new Chart(ctxR, {
            type: 'line',
```

```
                data: {
                    labels: labels,
                    datasets: [{ label: 'Recompensa Média', data: rewards, borde
                }
            });

            const ctxE = document.getElementById('tdErrorChart').getContext('2d'
            tdErrorChart = new Chart(ctxE, {
                type: 'line',
                data: {
                    labels: labels,
                    datasets: [{ label: 'Erro TD Médio', data: errors, borderCol
                }
            });
        }

        function renderTable(qTable, states) {
            const table = document.getElementById('qTableDisplay');
            let html = '<thead><tr><th>Estado \\ Ação</th>';
            for (let i = 0; i < 10; i++) html += `<th>A${i}</th>`;
            html += '<th>Ação Ótima</th></tr></thead><tbody>';

            states.forEach((stateName, s) => {
                html += `<tr><td><strong>${stateName}</strong></td>`;
                const maxVal = Math.max(...qTable[s]);
                const bestAction = qTable[s].indexOf(maxVal);

                for (let a = 0; a < 10; a++) {
                    const val = qTable[s][a].toFixed(1);
                    const isMax = (a === bestAction) ? 'class="highlight"' : '';
                    html += `<td ${isMax}>${val}</td>`;
                }
                html += `<td class="highlight"><strong>A${bestAction}</strong></
            });

            html += '</tbody>';
            table.innerHTML = html;
        }
    </script>
</body>
</html>
```

C. Estilização (`style.css`)

```
body {
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-ser
    margin: 0;
    padding: 0;
    background-color: #f7fafc;
    color: #2d3748;
}

header {
    background-color: #2b6cb0;
```

```
    color: white;
    padding: 1.5rem 2rem;
    text-align: center;
}

.container {
    max-width: 1200px;
    margin: 2rem auto;
    padding: 0 1rem;
}

.controls {
    background: white;
    padding: 1.5rem;
    border-radius: 8px;
    box-shadow: 0 2px 4px rgba(0,0,0,0.05);
    margin-bottom: 2rem;
}

.control-group {
    display: inline-block;
    margin-right: 1.5rem;
    margin-bottom: 1rem;
}

input[type="number"] {
    padding: 0.4rem;
    border: 1px solid #cbd5e0;
    border-radius: 4px;
    width: 90px;
}

button {
    background: #319795;
    color: white;
    padding: 0.6rem 1.2rem;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
}

button:hover {
    background: #285e61;
}

.status {
    margin-top: 0.8rem;
    font-style: italic;
    color: #4a5568;
}

.charts-grid {
    display: grid;
```

```
    grid-template-columns: 1fr 1fr;
    gap: 1.5rem;
    margin-bottom: 2rem;
}

.chart-card {
    background: white;
    padding: 1.5rem;
    border-radius: 8px;
    box-shadow: 0 2px 4px rgba(0,0,0,0.05);
}

.results {
    background: white;
    padding: 1.5rem;
    border-radius: 8px;
    box-shadow: 0 2px 4px rgba(0,0,0,0.05);
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 1rem;
    text-align: center;
}

th, td {
    padding: 0.6rem;
    border: 1px solid #e2e8f0;
}

th {
    background-color: #edf2f7;
}

.highlight {
    background-color: #c6f6d5;
    font-weight: bold;
}
```

3. Pipeline CI/CD para GitHub Pages (`.github/workflows/deploy.yml`)

Para garantir conformidade com as boas práticas de Engenharia de Software e MLOps, crie este fluxo de entrega contínua para publicar a simulação toda vez que você realizar um `push` na branch `main`:

```
name: Deploy GitHub Pages

on:
  push:
    branches:
      - main

permissions:
  contents: read
```

```
  pages: write
  id-token: write

concurrency:
  group: "pages"
  cancel-in-progress: true

jobs:
  deploy:
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Código
        uses: actions/checkout@v4
      - name: Configurar GitHub Pages
        uses: actions/configure-pages@v4
      - name: Upload de Artefatos
        uses: actions/upload-pages-artifact@v3
        with:
          path: '.'
      - name: Deploy para GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

4. Como Executar no Antigravity IDE e Subir para o GitHub

1. **Testar Localmente**:
   - Abra a pasta do projeto no **Antigravity IDE**.
   - Abra o arquivo `index.html` em um navegador padrão (ou utilize uma extensão de Live Server integrada).
   - Clique em **"Executar Simulação"** e veja as curvas convergindo e a Tabela 1 sendo preenchida conforme descrito no seu TCC.
2. **Inicializar o Repositório Git Local e Vincular ao GitHub**: No terminal do Antigravity IDE:

```
git init
git add .
git commit -m "feat: simulador do sistema tutor inteligente em javascript e
git branch -M main
git remote add origin https://github.com/SEU_USUARIO/NOME_DO_REPOSITORIO.git
git push -u origin main
```

3. **Ativar o GitHub Pages nas Configurações do Repositório**:
   - No GitHub, acesse: **Settings** → **Pages**.
   - Em **Build and deployment** / **Source**, selecione **GitHub Actions**.
   - O pipeline `.github/workflows/deploy.yml` será disparado automaticamente e criará a URL pública do seu projeto: `https://SEU_USUARIO.github.io/NOME_DO_REPOSITORIO/`.

Deseja que eu redija o modelo completo do `README.md` **acadêmico** para estruturar a documentação científica desse repositório de acordo com o padrão exigido na pós-graduação?

User prompt: sim, faça o readme

Response: Documentação do Repositório (README.md)

```
# Sistema Tutor Inteligente Adaptativo com Aprendizado por Reforço (Q-Learning)

[![GitHub Pages](https://img.shields.io/badge/GitHub%20Pages-Live%20Demo-brightg
[![License: MIT](https://img.shields.io/badge/License-MIT-blue.svg)](LICENSE)
[![Status: Concluído](https://img.shields.io/badge/Status-Concluído-success)](ht

Repositório oficial da simulação computacional desenvolvida no âmbito do Trabalh

* **Título do Trabalho:** *IA na Educação: Como o Aprendizado por Reforço (Reinf
* **Autor:** Harley Augusto Bergo
* **Ambiente de Simulação Online:** [https://SEU_USUARIO.github.io/sti-qlearning

---

## 1. Visão Geral

Softwares educacionais convencionais dependem tipicamente de trilhas estáticas l

O núcleo do tutor foi modelado sob a estrutura de **Processos de Decisão Markovi

---

## 2. Modelagem Matemática do Sistema

A interação entre o tutor (agente) e o estudante (ambiente) é formalizada pela q

### Espaço Discreto de Estados Cognitivos ($\mathcal{S}$)
O Módulo de Diagnóstico discretiza o perfil de retenção e autonomia do discente
* $E_0$ (**Ruim**): Retenção insuficiente, erros conceituais graves e necessidad
* $E_1$ (**Regular**): Compreensão conceitual preliminar, mas com hesitação elev
* $E_2$ (**Bom**): Domínio satisfatório dos fundamentos, resolução de problemas
* $E_3$ (**Muito Bom**): Autonomia consolidada e resolução consistente de proble
* $E_4$ (**Excelente**): Domínio pleno da ementa e prontidão para desafios conce

### Espaço de Ações Pedagógicas ($\mathcal{A}$)
O Modelo Pedagógico seleciona intervenções didáticas a partir de um conjunto de
* $A_0, A_1$: Revisão teórica passo a passo de conceitos basilares.
* $A_2, A_3$: Exemplos práticos resolvidos e comentados.
* $A_4, A_5, A_6$: Listas de exercícios de fixação com suporte gradual (*scaffol
* $A_7, A_8$: Problemas aplicados de nível médio-alto com mínima intervenção.
* $A_9$: Desafios avançados de síntese curricular.
```

Função de Recompensa ($\mathcal{R}$) e Dinâmica Estocástica
A cada transição completada, o ambiente emite uma recompensa escalar imediata at
$$r(E_0) = 1, \quad r(E_1) = 3, \quad r(E_2) = 5, \quad r(E_3) = 7, \quad r(E_4)

As probabilidades de transição modelam a **Zona de Desenvolvimento Proximal**: q

Regra de Atualização por Diferença Temporal (TD)
A Tabela Q de dimensão 5×10 tem suas células atualizadas a cada passo d
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a} Q

* **Taxa de aprendizado (α):** $0{,}20$
* **Fator de desconto temporal (γ):** $0{,}90$
* **Exploração vs. Explotação (ϵ-greedy):** $\epsilon_0 = 1{,}00$ com d

---

3. Resultados Experimentais Obtidos

Ao longo de 500 episódios simulados (20 passos por sessão), o agente exibe conve

| Métrica | Episódio 1 ($\epsilon = 0{,}985$) | Episódio 50 ($\epsilon = 0{,}470
| :--- | :---: | :---: | :---: | :---: |
| **Recompensa Média** | 2,60 | 4,75 | 9,45 | **9,55** |
| **Erro TD Médio Absoluto** | 3,07 | 9,60 | 0,75 | **0,70** |

Tabela Q Convergida (Q^*) após 500 Episódios

| Estado \ Ação | A_0 | A_1 | A_2 | A_3 | A_4 | A_5 | A_6 | A_7
| :--- | :---: | :---: | :---: | :---: | :---: | :---: | :---: | :---: | :---: |
| **E_0 (Ruim)** | **83,2** | 40,4 | 46,7 | 33,6 | 39,6 | 25,3 | 33,8 | 36,5 |
| **E_1 (Regular)** | 60,5 | 63,3 | **87,4** | 45,9 | 63,6 | 59,6 | 67,7 | 53,
| **E_2 (Bom)** | 70,0 | 65,3 | 67,8 | 68,9 | **92,6** | 64,6 | 70,7 | 70,2 |
| **E_3 (M. Bom)** | 86,8 | 87,3 | 84,6 | 88,6 | 88,0 | 89,3 | 88,3 | **96,1**
| **E_4 (Excelente)** | 91,7 | 92,3 | 92,5 | 92,6 | 93,2 | 93,1 | 91,8 | 92,8

---

4. Governança, MLOps e Implicações Éticas

O projeto incorpora os preceitos de sustentação e governança discutidos na monog
1. **Transparência e Explicabilidade (XAI):** A arquitetura tabular $5 \times 10
2. **Mitigação de Riscos de Exploração (*Warm Start*):** Destaca-se a obrigatori
3. **Engenharia de MLOps:** Versionamento contínuo de matrizes e parâmetros via

---

5. Estrutura do Repositório

```text
├── .github/
│   └── workflows/
│       └── deploy.yml        # Pipeline automatizado de deploy no GitHub Pages
├── index.html                # Interface gráfica do dashboard e painéis de cont
├── style.css                 # Folha de estilos e formatação visual
```

```
├── qlearning.js              # Implementação formal do ambiente PDM e do agente
├── README.md                 # Documentação técnica e científica do projeto
└── LICENSE                   # Licença MIT de código aberto
```

6. Como Executar Localmente

Pré-requisitos

- Qualquer navegador web moderno (Google Chrome, Firefox, Safari ou Edge).
- Git instalado.
- Editor de código (Antigravity IDE ou VS Code).

Instruções

1. Clone este repositório:

   ```
   git clone [https://github.com/SEU_USUARIO/sti-qlearning-simulation.git](http
   cd sti-qlearning-simulation
   ```

2. Abra o arquivo `index.html` diretamente no navegador ou utilize a extensão Live Server no Antigravity IDE:

   ```
   # Alternativa via Python (servidor local simples)
   python3 -m http.server 8000
   ```

3. Acesse `http://localhost:8000` no seu navegador.

4. Ajuste os parâmetros desejados nos campos de controle e clique em **Executar Simulação** para observar a convergência gráfica e o preenchimento matricial.

7. Referências Bibliográficas

- ARRIETA, Alejandro Barredo et al. Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. **Information Fusion**, v. 58, p. 82-115, 2020.
- BELLMAN, Richard. **Dynamic Programming**. Princeton: Princeton University Press, 1957.
- BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.
- KREUZBERGER, Dominik; KÜHL, Niklas; HIRSCHL, Sebastian. Machine Learning Operations (MLOps): Overview, Definition, and Architecture. **IEEE Access**, v. 11, p. 31866-31879, 2023.
- LEITE, A. S. **Um Modelo de Sistema Educativo Cognitivista Baseado em Tutoria Inteligente Adaptativa via Aderência Conceitual**. 1999. Tese (Doutorado) – Instituto Tecnológico de Aeronáutica, São José dos Campos, 1999.
- MARIETTO, Maria das Graças Bruno. **Definição Dinâmica de Estratégias Instrucionais em Sistemas de Tutoria Inteligente: Uma Abordagem Multiagentes na WWW**. 2000. Tese (Doutorado) – Instituto Tecnológico de Aeronáutica, São José dos Campos, 2000.

- MNIH, Volodymyr et al. Human-level control through deep reinforcement learning. **Nature**, v. 518, n. 7540, p. 529-533, 2015.
- SUTTON, Richard S.; BARTO, Andrew G. **Reinforcement Learning: An Introduction**. Cambridge: MIT Press, 1998.
- UNESCO. **Recommendation on the Ethics of Artificial Intelligence**. Paris: UNESCO, 2021.
- VICCARI, Rosa Maria. **Um Tutor Inteligente para a Programação em Lógica: Idealização, Projeto e Desenvolvimento**. 1990. Tese (Doutorado) – Universidade de Coimbra, Coimbra, 1990.
- WATKINS, Christopher J. C. H.; DAYAN, Peter. Q-learning. **Machine Learning**, v. 8, n. 3-4, p. 279-292, 1992.


```
---

Basta salvar esse conteúdo no arquivo `README.md` na raiz do seu projeto. Lembre
```
